

IMPERIUM OPERATIONALITY 70% WHOLE-REPO FUSION PLAN V0.1

English edition — aligned 1:1 in intent with the approved Russian plan

Purpose: transform IMPERIUM into a visible, controllable, observable, configurable, execution-capable operating system that can accept real work, perform it under gates, collect evidence, and produce deliverables ready for freelance/client use.

Current strategic frame: work now spans the whole IMPERIUM, not only IMPERIUM_TEST_VERSION. The system must fuse main canon, test technologies, organs, gates, scripts, agent profiles, Sanctum, Second Brain, and deliverable pathways into one coherent body.

Primary leitmotif

Do not build a decorative theory. Build a real operating machine that shows truth, accepts tasks, chooses a path, executes, warns, proves, packages, and helps earn money.

Global rule

Every new step must improve at least one of the following: **execution**, **truth**, **observability**, **control**, or **delivery**. If it does not, it should be deferred.

This plan intentionally stays large-scale and directional. It is designed to support adaptive corrections without losing the main operating thread.

1. Core goals and operational doctrine

Goal	Practical meaning
Execution capability	The system can accept a task and drive it to a deliverable.
Observability	The Owner sees state, stages, warnings, errors, receipts, diffs, memory/context, and progress per
Controllability	The Owner can stop, approve, reject, reroute, or re-scope work.
Configurability	Gates, scopes, task modes, output types, and risk levels can be configured.
Universality	The system supports many computer-based tasks, especially clean coding and working beautiful ap
Commercial usefulness	A freelance corridor exists: brief → questions → plan → execution → QA → delivery.
Visible incompleteness	The system honestly shows PASS, WARN, BLOCKED, UNKNOWN, and what remains unfinished.
Bilingual operation	Owner-facing interfaces and operational summaries must support both Russian and English, easily

Main law:

70% operational executability with honest truth is more valuable right now than 100% theoretical architectural perfection.

Adaptive correction rule

The Owner will continue introducing adaptive corrections while execution follows the main plan. These corrections must be captured and folded into roadmap, doctrine, or gates whenever they improve operational truth, control, or delivery.

Pain-to-law rule

Every repeated pain becomes a gate, policy, backlog item, checker, or registry field. This is already visible in GATE-U12 Report Output Budget and GATE-U13 Python Type Safety.

2. Whole-map roadmap — major stages

The stages below are ordered for operational efficiency. Each stage builds real leverage for the next. The plan works across the whole IMPERIUM body, not only one sub-tree.

Stage 0 — Whole-Imperium Truth Lock

Create a whole-repo reality map. Scan and classify the entire repository and surrounding active system zones into canon, test, tool, runtime, advisory, negative-sample, and dirty/reference categories. Identify useful absorption candidates, current blockers, duplication, and structural contradictions. This is the point where we stop treating isolated zones as separate worlds and establish one ground truth for the entire IMPERIUM body.

Stage 1 — Operationality 70 Doctrine and Roadmap

Register the doctrine that the immediate target is 70% real operational executability. Create a roadmap, readiness matrix, capability map, and readiness language: PASS, WARN, BLOCKED, UNKNOWN. This stage converts the Owner's strategic intent into a controlling doctrine for all future tasks.

Stage 2 — Gate Spine Expansion

Extend Gate Spine from the current local/Second Brain focus to whole-IMPERIUM use. Keep the existing gates and add new ones where reality demanded them: whole-repo scope recon, operational impact, bilingual UI, deliverable package, and agent-factory compliance. The purpose is to make large tasks safer, less wasteful, and more reusable.

Stage 3 — Scriptorium / Tool Absorption Spine

Inventory scripts and tools across main, test version, gate runners, Kiro/Codex-style outputs, and all practical coding zones. Group them into families, classify maturity levels, attach type-safety expectations, and decide what should be kept, hardened, absorbed, rewritten, or archived. This stage turns scattered coding gains into a true Mechanicus-owned tool system.

Stage 4 — Runtime Truth Baseline Completion

Finish the current Second Brain runtime truth line. Fix the runtime audit static-asset route issue, produce a valid full runtime baseline, classify asset budget, and ensure that UI performance claims are based on complete UI evidence rather than broken pages or partial backend-only truth. This stage closes the current blocker before visual expansion.

Stage 5 — Control Center Data Spine

Define the source contracts for a real control center. Specify where system state, task state, warnings, gates, receipts, scripts, capabilities, runtime status, agent states, and delivery readiness are read from. Build the data model first, before beauty. The purpose is to create a stable backbone for Sanctum + Second Brain operator panels.

Stage 6 — Sanctum + Second Brain Control Center MVP

Build the first real control center. The Owner must see progress, active task, states, errors, warnings, receipts, diffs, memory/context, commands, and capability status. The panel must be genuinely operational, not decorative. It should be bilingual by default and honest about what is unfinished.

Stage 7 — Imperium Zone Reality Map

Create the truth-bound map of zones: Doctrinarium, Administratum, Mechanicus, Inquisition, Officio Agentis, Astronomicon, Sanctum, Second Brain, Scriptorium, Arsenal, Freelance Corridor, Deliverable Factory, and Agent Factory. No zone may appear alive, green, or successful without source-backed evidence and valid receipts.

Stage 8 — Agent Factory Foundation

Design the agent-construction factory. Build a shared foundation and then many narrow micro-agents or “micro-neural blocks,” each with a narrow scope, allowed scripts, gates, receipts, and a visible output contract. Avoid the wasteful model of dozens of heavy general agents consuming limits without structure. Everything must be owned and observed from inside IMPERIUM.

Stage 9 — Freelance Corridor MVP

Create the first commercial task corridor: intake → clarification → plan → execution → QA → delivery. This gives the system its first practical money-making route. All stages must be visible, controllable, and evidence-backed.

Stage 10 — Deliverable Factory MVP

Create the factory that turns work into client-ready output. It must orchestrate code/app output, README creation, evidence screenshots, demo scenario writing, PowerPoint/presentation generation, package building, QA checklist creation, and client summary writing. This is where IMPERIUM becomes a freelance machine. It must be powered by a combination of local resources, hardware, scripts, internal LLMs/agents, cloud LLMs/agents, and optimization logic — not by imaginary “free magic.”

Stage 11 — Visual Construction

Only after truth, control, runtime baseline, and zone reality are in place do we build the cinematic visual layer. Create static or semi-static high-quality visual plates, overlay thin live truth, review through screenshot packets, and protect performance with real receipts. Beauty is built on proven truth, not instead of it.

Stage 12 — Whole-Imperium Fusion and Cleanup

Merge useful technology from test version into the whole body through controlled absorption. Normalize registries, path conventions, IDs, statuses, and migration receipts. Archive or quarantine negative samples. By the end of this stage, main canon, test technologies, tools, agents, and dashboards should feel like one body rather than fragments.

3. Recommended near-term sequence from the current point

To stay efficient and aligned with the goals, the immediate order should be the following:

Order	Move	Why now
1	Whole-Repo Operationality 70 Roadmap + Reality Scope	My Scope no longer working only in test zones; the entire IMPERIUM body is now in scope.
2	Whole-Repo Truth Inventory	We need one visibility map before more large work.
3	Scriptorium Tool Absorption Map	Useful code is spread across test/main/tool zones and must be classified.
4	Runtime Audit Static Asset Route Fix	Current full-runtime blocker must be removed.
5	Valid Full Runtime Baseline	We need real UI/API/FPS/load truth before visual claims.
6	Control Center Source Contract	The future dashboard needs a clean data spine.
7	Control Center MVP	The Owner needs a working panel as soon as possible.
8	Agent Factory Foundation	Micro-agents should be built on a shared base, not ad hoc.
9	Freelance Corridor MVP	This creates direct practical/commercial value.
10	Deliverable Factory MVP	Work must turn into packages, screenshots, and presentations.
11	Zone Reality Map	Now the whole body becomes visually legible.
12	Static Cinematic Visual Prototype	Beauty comes after truth-bound operations, not before.

The key efficiency principle is simple: first create one operational direction for the whole body, then improve execution paths, then improve visibility and delivery, and only after that invest in large-scale visual refinement.

4. First major task

Task ID:

TASK - IMPERIUM - OPERATIONALITY - 70 - WHOLE - REPO - FUSION - ROADMAP - V0_1

What this first step will do

This task will not attempt to fix the whole repository at once. Instead, it will create the unified operational map that lets every later task work in one direction. It should produce the first whole-repo fusion roadmap, readiness matrix, control-center MVP requirements, deliverable-factory contract, agent-factory foundation concept, bilingual UI policy, and new gate drafts required for this broader phase.

Why it is the right first step

Because we are now operating across the whole IMPERIUM, we must stop thinking in isolated local corridors. A unified map comes first. Once this map exists, large tasks can be executed with less waste, fewer blind spots, and far better coordination between organs, technologies, scripts, and future agents.

Target outputs

- ORGANS/ASTRONOMICON/ROADMAPS/IMPERIUM_OPERATIONALITY_70_WHOLE_REPO_FUSION_ROADMAP_V0_1.md/json
- ORGANS/ADMINISTRATUM/READINESS/OPERATIONALITY_70_READINESS_MATRIX_V0_1.md/json
- ORGANS/SANCTUM/CONTROL_CENTER/CONTROL_CENTER_MVP_REQUIREMENTS_V0_1.md/json
- ORGANS/SANCTUM/CONTROL_CENTER/BILINGUAL_UI_POLICY_V0_1.md/json
- ORGANS/ASTRONOMICON/FREELANCE_CORRIDOR/FREELANCE_CORRIDOR_MVP_CONTRACT_V0_1.md/json
- ORGANS/MECHANICUS/DELIVERABLE_FACTORY/DELIVERABLE_FACTORY_MVP_CONTRACT_V0_1.md/json
- ORGANS/OFFICIO_AGENTIS/AGENT_FACTORY/AGENT_FACTORY_FOUNDATION_CONCEPT_V0_1.md/json
- ORGANS/DOCTRINARIUM/GATES/WHOLE_REPO_SCOPE_RECON_GATE_V0_1.md/json (draft U14)
- ORGANS/DOCTRINARIUM/GATES/OPERATIONALITY_IMPACT_GATE_V0_1.md/json (draft U15)

- ORGANS/DOCTRINARIUM/GATES/BILINGUAL_UI_GATE_V0_1.md/json (draft U16)
- ORGANS/DOCTRINARIUM/GATES/DELIVERABLE_PACKAGE_GATE_V0_1.md/json (draft U17)
- ORGANS/DOCTRINARIUM/GATES/AGENT_FACTORY_COMPLIANCE_GATE_V0_1.md/json (draft U18)

Strict pass criteria for the first step

- **Whole-repo scope was genuinely considered.** The roadmap must explicitly reference main canon, test technologies, organs, scripts/tools, future agents, control center, freelance corridor, and deliverable factory.
- **Operationality 70 doctrine is explicit.** The resulting roadmap must clearly prioritize execution, truth, observability, control, and delivery.
- **Control-center requirements are concrete.** The panel must specify visible states, warnings/errors, progress, receipts, diffs, memory/context, commands, and bilingual support.
- **Deliverable factory is concrete.** The contract must explicitly include app builder, README builder, screenshot collector, demo scenario writer, PowerPoint builder, package builder, QA checklist builder, and client summary writer.
- **Agent-factory concept is concrete.** It must describe micro-agent profiles, script families, allowed tools, receipts, gates, and observational control under Sanctum + Second Brain.
- **New gate drafts exist.** The task must produce gate drafts for whole-repo scope recon, operational impact, bilingual UI, deliverable package, and agent-factory compliance.
- **No fake completeness.** The readiness matrix must show what is still WARN/BLOCKED/UNKNOWN.
- **Owner-facing outputs are bilingual-ready.** The requirements and doctrine must state RU/EN behavior explicitly.
- **Forbidden modifications are respected.** No runtime/UI source refactoring or unrelated large code changes occur in this task.
- **Evidence package exists.** Reports, action card, gate receipt, and self-assessment commentary are included.

Mandatory agent self-assessment requirement

Whenever an agent or servitor submits and commits work, the output bundle must include a self-assessment section inside the task files. For every pass criterion, the agent must provide an honest answer: whether it fully passed, partially passed, or did not pass, along with a short explanation and evidence path. This self-commentary is mandatory and should later fall under Officio Agentis response contracts.

Required structure of self-assessment comment:

```
criterion_id → verdict (PASS / PARTIAL / FAIL) → short explanation → evidence
path(s) → self-risk note, if any
```

5. Task-gate package for large tasks

Because the next tasks are large and strategic, they need explicit task gates that can later be absorbed into Officio Agentis. The package below is recommended for every large execution task.

Mandatory task gates

- **GATE-Task-Clean-Start** — no dirty worktree before execution.
- **GATE-Task-Scope-Recon** — the agent must inspect all relevant repository zones before editing.
- **GATE-Task-Forbidden-Paths** — critical paths cannot be changed unless the task explicitly allows it.
- **GATE-Task-Output-Budget** — reports must remain compact and actionable.
- **GATE-Task-Self-Assessment** — the agent must honestly grade itself against pass criteria.
- **GATE-Task-Receipt-Completeness** — action card, gate receipt, chronology report, and evidence paths must exist.
- **GATE-Task-Operational-Impact** — the task must improve execution, truth, control, observability, or delivery.

Forbidden conditions

- Fake green: a success claim without evidence.
- Hidden dirty state or hidden partial work.
- Oversized unreadable reports that defeat review.
- Silent deletion or move/rename of unrelated files.
- Source changes outside declared scope.
- A “complete” claim that does not honestly mark warnings or blocked items.

6. Chronology requirement

For large tasks, the final report must describe the work chronologically. Each step must have its own paragraph explaining what was done, why it was done, and how to use the result. This is required both for Owner understanding and for future agent memory/continuity.

- Step N title
- What was done
- Why it was necessary
- What artifact(s) were produced
- How to use or verify the result
- What remains unfinished or risky

7. How the system should feel after these stages

When these stages are implemented in order, IMPERIUM should begin to behave like a real operating body. The Owner can create and register tasks, see state and errors, receive the best known path, monitor execution, review receipts, collect screenshots, build presentations, package delivery artifacts, and eventually route diverse freelance work through one controlled machine. This is the path to the first practical earnings and to later modernization from a position of real strength.

Final guiding sentence

Build one body. Show truth. Execute visibly. Deliver cleanly. Grow through gates.